

Java プログラムの基礎

ver. 1.0

presented by t.ogisawa[®]

目次

0.	はじめに:開発環境の準備とプログラムの実行方法	3
0.1.	必要なもの:JDK	3
0.2.	プログラムを実行する2つの方法	3
0.2.1.	コマンドラインで実行する方法(基本)	3
0.2.2.	統合開発環境(IDE)で実行する方法(推奨)	3
1.	Javaプログラムの基礎	4
1.1.	main メソッド	4
1.2.	キーボードからの入力	5
2.	文(ステートメント:statement)	6
3.	データ型と変数(バリアブル:variables)	7
3.1.1.	データ型	7
3.1.2.	変数の宣言	8
3.1.3.	値の表現(リテラル:literal)	9
3.1.4.	定数(コンスタント:Constant)	9
3.1.5.	数値型の変換(キャスト:cast)	9
3.1.6.	数式による型変換	10
3.1.7.	文字列とエスケープシーケンス(escape sequence)	10
3.1.8.	配列(アレー:Array)	11
4.	識別子(identifier)	12
5.	演算子(オペレータ:operator)	13
5.1.	算術演算子	13
5.2.	関係演算子	13
5.3.	論理演算子	14
5.4.	代入演算子・複合演算子	14
5.5.	インクリメント・デクリメント演算子	14
5.6.	その他	15
5.7.	演算子の優先順位	15
6.	Javaの基本構文(シンタックス:Syntax)	17
6.1.	条件分岐(if文)	17
6.2.	条件分岐(switch文)	18
6.3.	繰り返し(while文)	20
6.4.	繰り返し(do文)	20
6.5.	繰り返し(for文)	20
6.6.	ループを抜ける(break文)	21
6.7.	ループを続ける(continue文)	21
7.	メソッド(method)	22

0. はじめに：開発環境の準備とプログラムの実行方法

この資料で Java プログラミングを学ぶ前に、書いたコードを動かすための準備について理解しておきましょう。

0.1. 必要なもの：JDK

Java プログラムを動かすには、JDK (Java Development Kit) という開発ツールが必要です。JDK には、人間の書いたソースコード (.java ファイル) をコンピュータが理解できる形式 (.class ファイル) に変換するコンパイラと、そのファイルを実際に実行する実行環境が含まれています。

0.2. プログラムを実行する 2 つの方法

0.2.1. コマンドラインで実行する方法（基本）

- ・ソースコードを書く：
テキストエディタでコードを書き、App.java のようにクラス名.java という名前で保存します。
- ・コンパイルする：
コマンドプロンプトやターミナルで、ファイルの場所に移動し、以下のコマンドを実行します。

```
javac App.java
```

成功すると、App.class というファイルが生成されます。これがコンピュータが実行するファイルです。

- ・実行する：
以下のコマンドでプログラムを実行します。.class は付けないことに注意してください。

```
java App
```

0.2.2. 統合開発環境（IDE）で実行する方法（推奨）

実際の開発では、IDE (Integrated Development Environment) という高機能な開発用ソフトウェアを使うのが一般的です。

IDE とは？: コードを書くためのエディタ、コンパイラ、デバッグ (バグを見つけるツール) などが一つにまとまった、プログラミング専用の強力な道具です。

主な IDE: Eclipse、IntelliJ IDEA、Visual Studio Code などがあります。

メリット: IDE を使えば、ボタン一つでコンパイルから実行までを自動で行ってくれるため、開発に集中できます。本研修でも IDE の利用を推奨します。

1. Java プログラムの基礎

1.1. main メソッド

最初に呼ばれるソースプログラムを以下に示します。

“App.java”というファイルを作ってください。このファイル名は、1行目のクラス名:“App”に合わせて+拡張子:“.java”を付けたものです。

```
1: public class App {  
2:     public static void main(String[] args) {  
3:         System.out.println("Hello World");  
4:     }  
5: }
```

Hello World

public (パブリック)

「プログラムのどこからでも使うことができる」という意味です。

class (クラス)

Java はオブジェクト指向言語であるため、全てのプログラムは部品（モノ）単位で記述し class というキーワードを使って部品を定義します。

「class クラス名」で任意のクラス名を指定できます。class の範囲は{}で囲みます、この範囲をブロックといいます。1行目から5行目までが class のブロックです。

main メソッド

クラスにはメソッド（モノを操作する方法）を定義できます。

main メソッドは Java プログラムを実行したときに、最初に自動的に呼び出される特別なメソッドです。

public や static といったキーワードについて、詳細はオブジェクト指向の章で学びます。

System.out.println

Java の標準機能で System.out.println メソッドです。

ここでは、表示のための命令が1行あります。System.out.println("Hello World");

1.2. キーボードからの入力

プログラム内に書かれたデータを表示するだけでなく、Java の標準機能である `Scanner` クラスを使いプログラム実行中にユーザーがキーボードから入力した値を受け取ることもできます。

以下にユーザーからの入力値を表示する例を示します。

```
import java.util.Scanner; // 1. Scanner を使うためのおまじない

public class InputExample {
    public static void main(String[] args) {
        // 2. キーボード入力を受け取る準備
        Scanner scanner = new Scanner(System.in);

        // 文字列の入力を受け取る
        System.out.print("あなたの名前を入力してください: ");
        String name = scanner.nextLine();

        // 整数の入力を受け取る
        System.out.print("あなたの年齢を入力してください: ");
        int age = scanner.nextInt();

        // 結果を表示する
        System.out.println("こんにちは、" + name + "さん！ あなたは" + age + "歳なのですね。");
    }
}
```

2. 文（ステートメント：statement）

Java のプログラムは『文（ステートメント）』の集まりで構成され、文の終わりにはセミコロン (;) を付けます。ステートメントは、以下の構成要素から成り立ちます。

値、演算子、式、キーワード、コメント

値（バリュー：value）とは、プログラムのソースコードにおいて使用される、数値や文字列を直接に記述した定数（リテラル：literal）と、変えることのできる変数（バリアブル：variables）です。

通常値は、変数を定義して定数を代入し使います。

例：

```
int a; //変数の定義文
String str;
a = 1+2; //変数の代入文
str = "my work";
```

演算子（オペレーション：operation）は、変数に値を代入や計算に使われます。

例： = + - * / % ()

```
int a = 1;
int b = 2;
a = (a + b - 1) * 2 / 2
```

式（エクスプレッション：expression）は、演算子により計算される文の一部です。

例：

```
1+2
"Jone live in tokyo"+" with satoko "
```

コメント（comment）は、プログラムの理解等のメモに使うコミュニケーション記述です。

例：// /* ~ */

```
int a = 1; //これがコメント
/* これもコメント
複数行に跨って記述できます */
```

3. データ型と変数（バリアブル：variables）

プログラムが動作するとき、メモリにさまざまな値（バリュー：value）を記憶させながら処理をします。プログラムは、必要な分だけ値の入れ物を用意して処理を行います。その入れ物のことを変数（バリアブル：variables）と呼びそれを扱うために名前をつけ値を入れたり参照したり違う値に更新したりします。

プログラムが扱う値には色々なタイプがあります、数値であったり文字列であったりします。
Java では変数を定義するときに値を入れるデータの型を定義します。

3.1.1. データ型

基本データ型

種類	データ型	サイズ	値の範囲
整数	byte	8 ビット (1 バイト)	-128 ~ +127
	short	16 ビット (2 バイト)	-32768 ~ +32767
	int	32 ビット (4 バイト)	-2147483648 ~ +2147483647
	long	64 ビット (8 バイト)	-9223372036854775808 ~ +9223372036854775807
浮動小数点	float		32 ビット (4 バイト) ±3.40282347E+38 ~ ±1.40239846E-45
	double	64 ビット (8 バイト)	±1.79769313486231570E+308 ~ ±4.94065645841246544E-324
文字	char	16 ビット (2 バイト)	0 ~ 65535, 'a', 'a', 'あ'...
真偽	boolean	(JVM の実装による)	true or false

参照型

文字列	String	文字列長による	null or “xxxxxx...”
-----	--------	---------	---------------------

3.1.2. 変数の宣言

```
int myNum;    // 変数宣言
myNum = 10;   // 変数に値を入れる
System.out.println(myNum);

int myNum = 15; // 変数宣言と同時に値を設定する
System.out.println(myNum);

String name = "John";
System.out.println(name);

long l = 123456789012L;
float f = 3.14f;
double d = 3.14
char ch = 'D'; //一文字を代入するシングルクォーテーション
```

Java の内部文字コードは、Unicode (UTF-16)です。

```
char chA = 'A';
char ch41 = 0x41;
System.out.println("chA =" + chA + " vs 0x41=" + ch41); // chA=A vs 0x41=A

char chB = 'あ';
char ch3042 = 0x3042;
System.out.println("chB =" + chB + " vs 0x3042=" + ch3042); // hB=あ vs 0x3042=あ
boolean b = true;
String str = "Hello";

// 複数変数宣言
int x = 5, y = 6, z = 50;
int x, y, z;
x = y = z = 50;
```

宣言した変数が見える範囲はなんらかの{~}までの中括弧の範囲で、それをブロックまたはコードブロックと呼びます。

```
{
    int x = 123;
    System.out.println("x = " + x);
}
System.out.println("x = " + x);    // コンパイルエラーになる
```


3.1.3. 値の表現（リテラル：literal）

数値や文字列を直接に記述した値そのものを、リテラル：literal といいます。

変数を定義して定数を代入し使います。

単純に数値を記述した場合は 10 進数の整数となり、int 型（4 バイト）として扱われます。

123456789012 は、4 バイトを超えるのでその様な値は、末尾に'L'をつけて long 型(8 バイト)の整数であることを明示します。

小数点の数値を記述すると double 型(8 バイト)の領域に確保されます。

従って、3.14 のリテラルも double 型となり float f = 3.14 書いても、float 型(4 バイト)の変数には入らないため、末尾に'F'を付けて float 型の浮動小数点数であることを明示します。

3.1.4. 定数(コンスタント：Constant)

変数に値を設定して変更しないものには、final キーワードを使って定義することによって間違えて変更できないようにします。

```
final float PI = 3.14;  
final int DAYS_IN_WEEK = 7;
```

3.1.5. 数値型の変換（キャスト：cast）

表現の大きな型に小さな型の値を代入する場合は、自動的に変換されて代入されます。

```
int i = 10;  
long l = i;  
float f = l;  
double d = f;
```

小さな型変数入り切らない程の大きな値を、無理に小さな型変数に入れると差分の上位バイトの値が切り捨てられ不明な値が代入されます。これは「オーバーフロー」や「桁あふれ」と呼ばれる現象で注意が必要です。

現在大きな型変数に入っているが値が、例え小さいと分かっている、それを小さな型変数に移し替えるには、キャストと呼ばれる記法が必要です。

```
long l = 10;  
int a = (int)l;  
short b = (short)l;
```

3.1.6. 数式による型変換

```
int a = 100;
double b = 2.0
double c = a / b;
```

異なる型の演算では、表現能力の大きい方の型に合わせて演算されます。
int 型の a が、演算時に内部的に double に拡張変換（算術昇格）されて、100.0 / 2.0 の演算が行われるので、結果を代入する変数 c のデータ型は double 型にする必要があります。

Java の演算時における規則には、以下があります。

- char, byte, short 型は、演算の前に int に変換される
- 整数同士・浮動小数点同士で型が異なる場合は、バイトの大きい方に合わせられる
- 2つのデータ型が同じ場合は、演算結果もそのデータ型になる
- 浮動小数点と整数では、浮動小数点に合わせられる

3.1.7. 文字列とエスケープシーケンス(escape sequence)

Java で文字列を扱うには下記のように参照型の String を使います。

```
String str = "This is my home town!"
```

文字列中に、改行やタブなど特別な文字を指定したい場合、エスケープシーケンスを使って表現します。

エスケープシーケンス

種別	説明	使用例
¥n	改行コード	String str = "Thank you!¥n"
¥t	タブ文字	String str = "age: ¥t20"
¥¥	バックスラッシュ	String str = "¥¥1,000"
¥'	シングルクォーテーション (')	char c = '¥'
¥"	ダブルクォーテーション (")	String str = "My name is ¥"Aska¥""
¥uxxxx	Unicode xxxx の文字	"¥u3042"

3.1.8. 配列（アレー：Array）

配列は、同じ型の複数データを扱うための変数形式です。

配列変数を使った例を下記に示します。

```
int[] myNum = {10, 20, 30, 40};
int[] myNum;
myNum = {0, 20, 30, 40};           // エラー
myNum = new int[]{0, 20, 30, 40};   // OK!

String[] cities;
String[] cities = {"New York", "London", "Paris", "Tokyo"};
cities [0] = "Boston";  // 0 番目の要素
System.out.println(cities [0]);

// for 文でインデックスでのループ
for (int i = 0; i < cities.length; i++) {
    System.out.println(cities [i]);
}
// for-each 文でのループ
for (String city : cities) {
    System.out.println(city);
}
// 多重配列
int[][] myNumbers = { {1, 2, 3}, {11, 12, 13} };
First row: myNumbers[0]は、{1, 2, 3}
Second row: myNumbers[1]は、{11, 12, 13}
for (int i = 0; i < myNumbers.length; i++) {
    for (int j = 0; j < myNumbers[i].length; j++) {
        System.out.println(myNumbers [i][j]);
    }
}
```

配列の範囲を超えた添字（インデックス）で、配列要素をアクセスすると

java.lang.ArrayIndexOutOfBoundsException というエラーが発生して終了してしまいます。

```
String[] cities = {"New York", "London", "Paris", "Tokyo"};
System.out.println(cities[5]);

Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 5
out of bounds for length 5.
```

4. 識別子 (identifier)

識別子 (identifier) とは、変数 (・メソッド・ラベル) 等で使う名称のことです。

使える文字は、半角アルファベット、半角数字、'\$'、'_'です、全角を使ってはいけません。

一文字目に、半角数字は使えません。尚、半角アルファベットは、大文字小文字を区別します。

例えば変数"int myName;"と"int myname;"を定義したら別の変数となります。

よく使う識別子の名前づけルールは、次の2つのいずれかです。

- ・スネークケース：単語の区切りをアンダースコアで繋げる

例：first_name, last_name, master_key, our_city

- ・キャメルケース：単語の先頭を大文字にする

例：FirstName, LastName, MasterKey, OurCity

firstName, lastName, masterCard, tokyoCity

どちらの記法を用いるかはプログラミング言語によって異なりますが、Java では通常キャメルケースでコーディングします。但し、定数は通常の変数と区別するため大文字のスネークケースで記述します。

```
PI // 円周率定数
```

ここでは簡単のため、int a;や int l;等の無意味な変数名を使いますが実際のプロジェクトでは変数名や関数名を見ただけで分かるように意味ある名前を付けましょう！

型の種類や意味を、小文字で先頭につけることも良くあります。例えば次のようなものです。

```
numCount    // 数字のカウント
indexCount   // インデックスのカウント
doubleCount  // ダブル型定数の入るカウント
idUser       // ユーザ ID
iUser        // ユーザのインデックス
nameUser     // ユーザ名
```

5. 演算子（オペレータ: operator）

値を計算したり比べたり計算処理をするときに用いる記号を演算子（オペレータ: operator）と呼びます。
Java の演算子には、算術演算子、関係演算子（比較演算子）、論理演算子、代入演算子があります。

5.1. 算術演算子

演算子	名称	使用例	説 明
+	加算・連結	<code>x + y</code>	<code>x</code> と <code>y</code> を足し合わせる(加算) 文字列の場合は文字列末尾への連結
-	減算	<code>x - y</code>	<code>x</code> から <code>y</code> を引く
*	乗算	<code>x * y</code>	<code>x</code> と <code>y</code> を掛け合わせる
/	除算	<code>x / y</code>	<code>x</code> を <code>y</code> で割る
%	剰余算	<code>x % y</code>	<code>x</code> を <code>y</code> で割った余りを求める

5.2. 関係演算子

演算子	名称	使用例	説 明
<code>==</code>	同等比較	<code>x == y</code>	<code>x</code> と <code>y</code> が等しい場合 <code>true</code>
<code>!=</code>	非同等比較	<code>x != y</code>	<code>x</code> と <code>y</code> が等しくない場合 <code>true</code>
<code>></code>	大比較	<code>x > y</code>	<code>x</code> が <code>y</code> よりも大きい場合 <code>true</code>
<code><</code>	小比較	<code>x < y</code>	<code>x</code> が <code>y</code> より小さい場合 <code>true</code>
<code>>=</code>	大 比較	<code>x >= y</code>	<code>x</code> が <code>y</code> より大きいか等しい場合 <code>true</code>
<code><=</code>	小 比較	<code>x <= y</code>	<code>x</code> が <code>y</code> より小さいか等しい場合 <code>true</code>
<code>?</code>	三項演算子 (<code>? :</code>)	<code>b == true ? x : y</code>	関係演算子結果などを使った式が、 <code>true</code> だったら <code>x</code> 、 <code>false</code> だったら <code>y</code> を返す

5.3. 論理演算子

演算子	名称	使用例	説 明
&&	論理積	x && y	x と y の両方が true の場合に true
	論理和	x y	x と y のいずれかが true の場合に true
!	論理否定	!x	x が false の場合に true（真偽値を逆転）

5.4. 代入演算子・複合演算子

演算子	名称	例	同意例
=	代入	x = y	x = y
+=	加算代入	x += y	x = x + y
-=	減算代入	x -= y	x = x - y
*=	乗算代入	x *= y	x = x * y
/=	除算代入	x /= y	x = x / y
%=	剰余算代入	x %= y	x = x % y

5.5. インクリメント・デクリメント演算子

演算子	名称	例	同意例
++ （前置）	インクリメント(前置)	++x	x に 1 を足した後に式を評価
++ （後置）	インクリメント(後置)	x++	式を評価した後に 1 を足す
-- （前置）	デクリメント(前置)	--x	x から 1 を引いた後に式を評価
-- （後置）	デクリメント(後置)	x--	式を評価した後に 1 を引く

5.6. その他

演算子	説 明	例
()	算術グループ	x = (y + z) * 2
.	オブジェクトプロパティ	person.name
[]	配列インデックス	array[3]
()	メソッド呼び出し	aMethod()
new	オブジェクト作成	new Person
instanceof	オブジェクトのクラス取得	instanceof x

5.7. 演算子の優先順位

演算子の優先順位は以下ですが、計算順序を明示する方が安全で分かりやすいので複雑な式では積極的に()を使って下さい。

優先順位	演算子
↑ 高い	配列 ([]) 括弧 (()), 後置き (++ --)
	単項演算子(! ~ + -), 前置き (++ --)
	new、(キャスト型)
	乗除余 (* / %)
	加減 (+ -) 文 字列連結 (+)
	シフト (<< >> <<<)
	比較 (< <= >= >) instanceof
	比較 (== != === !==)
	AND (&)
	XOR (^)
↓ 低い	OR ()
	かつ (&&)

または (||)

三項演算子 (? :)

代入 (=) 複合代入 (+= -= *= /= %= <<= >>= >>>= &= |= ^=)

優先順位を変えることのできる括弧が最も強く、次に、算術演算子、関係演算子、論理演算子の順となります。代入演算子は最も弱くなります。

6. Java の基本構文（シンタックス：Syntax）

構文（syntax）は、プログラムがどのように書かれるべきかを定義するルールの集まりを指し、プログラミング言語の基本的な構造や文の組み立て方を示します。

キーワード

キーワード	説 明
break	switch 文とループ処理のブレイク
continue	ループ処理における中断と先頭へのジャンプ
do ... while	ブレイクするまでのループ処理
for	条件付きループ処理
if ... else	条件分岐
return	メソッドからの戻り
switch	変数の複数の条件分岐

Java の基礎構文を下記に見ていきましょう。

6.1. 条件分岐（if 文）

if (expression) statements1 else statements2

expression が真であれば statements1 を、偽であれば statements2 を実行します。例えば次の例では、変数 n の値が 10 より小さければ、SMALL を、偽であれば BIG を表示します。

```
if (n < 10) {
    System.out.println("SMALL");
} else {
    System.out.println("BIG");
}
// else 以降は記述しない場合もあります。
if (n < 10) {
    System.out.println("SMALL");
}
```

次のような書き方もできます。

```
if (n < 10) {
    System.out.println("SMALL"); // 10 より小さければ SMALL を表示
} else if (n > 20) {
    System.out.println("BIG");    // 20 より大きければ BIG を表示
} else {
    System.out.println("NORMAL"); // それ以外 NORMAL を表示
}
```

6.2. 条件分岐 (switch 文)

```
switch (expression) { case value: statements; ... }
```

expression の値に応じて処理を振り分けます。次の例では、xx の値が 1 なら ONE を、2 なら TWO を、3 か 4 なら THREE or FOUR を、それ以外なら OTHER を表示します。

```
switch (xx) {
case 1:
    System.out.println("ONE");
    break;
case 2:
    System.out.println("TWO");
    break;
case 3:
case 4:
    System.out.println("THREE or FOUR");
    break;
default:
    System.out.println("OTHER");
    break;
}
```

enum を使った例を示します。

```
public enum WeekDay { SUNDAY, MONDAY, TUESDAY,
    WEDNESDAY, THURSDAY, FRIDAY, SATURDAY; }

int numLetters = 0;
WeekDay day = WeekDay.WEDNESDAY;
switch (day) {
    case MONDAY:
```

```

case FRIDAY:
case SUNDAY:
    numLetters = 6;
    break;
case TUESDAY:
    numLetters = 7;
    break;
case THURSDAY:
case SATURDAY:
    numLetters = 8;
    break;
case WEDNESDAY:
    numLetters = 9;
    break;
default:
    throw new IllegalStateException("Invalid day: " + day);
}
System.out.println(numLetters);

```

switch 文で String 型がサポートされています。

```

String fruit = "orange";
switch (fruit) {
    case "apple":
        System.out.println("apple");
        break;
    case "orange":
        System.out.println("orange");
        break;
    default:
        System.out.println("Unknown ? ");
}

```

尚、Java14 から以下の様なシンプルな記述の出来る switch 式もサポートされました。

```

String fruit = "orange";
switch (fruit) {
    case "apple" -> System.out.println("apple");
    case "orange" -> System.out.println("orange");
    default -> System.out.println("Unknown ? ");
}

```

6.3. 繰り返し (while 文)

`while (expression) statements`

`expression` が真である間、`statements` を繰り返します。次の例では 0123456789 と表示されます。

```
int n = 0;
while (n < 10) {
    System.out.println(n);
    n++;
}
```

6.4. 繰り返し (do 文)

`do statements while (expression)`

`expression` が真である間、`statements` を繰り返します。次の例では 0123456789 と表示されます。最初から `expression` が偽であっても、`statements` が少なくとも 1 回は実行される点が `while (...) {...}` と異なります。

`int n = 0;`

```
do {
    System.out.println(n);
    n++;
} while (n < 10);
```

6.5. 繰り返し (for 文)

`for (expression1; expression2; expression3) statements`

最初に `expression1` を実行し、次に、`statements` と `expression3` を `expression2` が真である間繰り返します。次の例は 10 回繰り返す処理の典型的な例で、0123456789 と表示されます。

```
for (int i = 0; i < 10; i++) {
    System.out.println(i);
}
```

拡張 for 文： `for-each statements`

`for (type variableName : arrayName)`

配列のすべての要素に関する処理を行います。次の例では ABC が表示されます。

```
char [] array= new char[3];
array [0] = 'A';
```

```
array [1] = 'B';  
array [2] = 'C';  
for (char ch : array) {  
    System.out.println(ch);  
}
```

6.6. ループを抜ける (break 文)

break は、最も内側の for、while、do ループや、switch 文の case 節を抜けます。次の例では、01234 と表示されます。

```
for (int i = 0; i < 10; i++) {  
    if (i == 5) {  
        break;  
    }  
    System.out.println(i);  
}
```

6.7. ループを続ける (continue 文)

ループ内の残りの処理をスキップし、もっとも内側の for、while、do ループの次のループを実行します。次の例では 5 がスキップされて 012346789 と表示されます。

```
for (int i = 0; i < 10; i++) {  
    if (i == 5) {  
        continue;  
    }  
    System.out.println(i);  
}
```

7. メソッド (method)

メソッドとは、ある特定の処理を呼び出して実行できる名前付きの処理単位（コードブロック）です。メソッドを定義しておけば、何度でも呼び出して実行できるので、同様コードを書かなくても済みます。

メソッドを呼び出す際には、パラメータとしてデータを呼び出し元から渡すことができます。これを引数とも呼びます。渡す引数は複数で構いません。メソッドは、渡されたパラメータに応じて処理を行います。

また、メソッドは処理の結果を呼び出し元に返すこともできます。この返される値を戻り値（または返り値と呼びます）。

この様に、メソッド化によりコードの重複をなくしプログラム全体の可読性と保守性を向上させることができます。

```
public class Main {  
    static void myMethod() {  
        System.out.println("I just got executed!");  
    }  
  
    static int plusInt(int x, int y) {  
        return x + y;  
    }  
  
    static double plusDouble(double x, double y) {  
        return x + y;  
    }  
  
    static String plusStr(String x, String y) {  
        return x + y;  
    }  
  
    static int sum(int k) {  
        if (k > 0) {  
            return k + sum(k - 1);  
        } else {  
            return 0;  
        }  
    }  
  
    public static void main(String[] args) {  
        myMethod();  
        int myInt = plusInt(2, 5);  
        double myDouble = plusDouble(2.3, 5.1);  
        String myStr = plusStr("This is ", "a apple.");  
        System.out.println("int: " + myInt);  
        System.out.println("double: " + myDouble);  
    }  
}
```

```
    System.out.println("String: " + myStr);  
  }  
}
```